Working with R and Julia tools

BayesForge does not replace the data-manipulation and post-processing tools of your language. Data go in as the data frames you already use, and posterior draws come out as native R or Julia objects, so you can keep using dplyr, ggplot2, posterior, bayesplot, loo, DataFrames.jl or MCMCChains.jl.

Data in: pass your data frame

`m.data()` accepts a file path **or** an in-memory table. Manipulate your data in your own language, then hand it over; nothing needs to be written to disk.

Python

```python
import pandas as pd
from BayesForge import bf

m = bf(platform="cpu", rand_seed=1)
df = pd.read_csv(m.load.howell1(only_path=True), sep=";")
df = df[df.age > 18]          # any pandas manipulation
m.data(df)                    # a pandas DataFrame
m.scale(["weight"])
m.data_to_model(["weight", "height"])
```

R

```r
library(BayesForge)
m <- importBF(platform = "cpu", rand_seed = 1)

df <- read.csv(m$load$howell1(only_path = TRUE), sep = ";")
```

```r
df <- subset(df, age > 18)    # any R manipulation (dplyr, data.table, ...)
m$data(df)                    # an R data.frame
m$scale(list("weight"))
m$data_to_model(list("weight", "height"))
```

Julia

```julia
using PyBayesForge, PythonCall, DataFrames, CSV

m = importBF(platform = "cpu", rand_seed = 1)
path = pyconvert(String, m.load.howell1(only_path = true))
df = CSV.read(path, DataFrame; delim = ';')
df = filter(:age => >(18), df)  # any DataFrames.jl manipulation
m.data(df)                       # a Julia DataFrame (any Tables.jl table)
m.scale(["weight"])
m.data_to_model(["weight", "height"])
```

Draws out: native posterior objects

After `fit()`, the posterior draws can be extracted as a table with one row per draw: `.chain`, `.iteration`, `.draw` (R) or `chain`, `iteration`, `draw` (Julia), then one column per scalar parameter (`a[1]`, `a[2]`, … for vectors).

Python

```python
import arviz as az

def model(weight, height):
    a = m.dist.normal(178, 20, name="a")
    b = m.dist.log_normal(0, 1, name="b")
    s = m.dist.uniform(0, 50, name="s")
    m.dist.normal(a + b * weight, s, obs=height)

m.fit(model)

m.posteriors                    # dict of arrays (draws, ...)
idata = az.from_numpyro(m.sampler)   # ArviZ InferenceData
```

R

```r
model <- function(weight, height) {
    a <- bf.dist.normal(178, 20, name = "a")
    b <- bf.dist.log_normal(0, 1, name = "b")
    s <- bf.dist.uniform(0, 50, name = "s")
    bf.dist.normal(a + b * weight, s, obs = height)
}
m$fit(model)

draws <- bf_draws(m)            # a plain data.frame
head(draws)

# posterior / bayesplot / loo ecosystem
draws_df <- posterior::as_draws_df(m)
posterior::summarise_draws(draws_df)
bayesplot::mcmc_areas(draws_df, pars = c("b"))

# brms object, when the model has a brms equivalent
# fit_brms <- to_brms(m$sampler, formula = height ~ weight, data = df)
```

Julia

```julia
using DataFrames, MCMCChains

@BF function model(weight, height)
    a = m.dist.normal(178, 20, name = "a")
    b = m.dist.log_normal(0, 1, name = "b")
    s = m.dist.uniform(0, 50, name = "s")
    m.dist.normal(a + b * weight, s, obs = height)
end
m.fit(model)

draws = DataFrame(bf_draws(m))  # a Julia DataFrame
chains = Chains(m)              # an MCMCChains.Chains object
describe(chains)
```

Indexing inside model code is 0-based

Model code is executed by Python (JAX), even when written in R or Julia, so indexing a parameter inside the model is **0-based**: `a[0]` is the first element. JAX does not raise an error for an out-of-range index — it silently returns the last element — so `a[2]` on a parameter of length 2 is a bug that runs. Draw tables follow your language's convention: `a[0]` in the model is the column `a[1]` in `bf_draws()`.

Note

The language interfaces are thin layers over the same Python engine: R uses **reticulate** and Julia uses **PythonCall**. Model definitions therefore follow the Python API (`m.dist.normal(...)`, `name=`, `obs=`), while data handling and post-processing stay in your language.